

Guide de Travail

Workflow Git & Organisation

Projet Chatbot SUP'PTIC

Comment travailler efficacement en équipe
avec Git, les branches et les Pull Requests

Club Informatique SUP'PTIC
Février 2026

Table des matières

1	Organisation du Dépôt Git	3
1.1	Structure des Branches	3
1.2	Branches de Travail par Équipe	3
2	Règles du Dépôt	4
2.1	Règle #1 : Ne touchez pas à main et dev directement	4
2.2	Règle #2 : Travaillez sur vos propres branches	4
2.3	Règle #3 : Créez une Pull Request (PR) quand vous êtes prêts	4
3	Workflow Résumé pour l'Équipe	5
4	Démarrer : Cloner le Dépôt	6
4.1	Étape 1 : Cloner le Dépôt sur Votre Machine	6
4.2	Étape 2 : Vérifier les Branches Disponibles	6
5	Récupérer Votre Branche de Travail	7
5.1	Cas 1 : Vous êtes dans l'Équipe Frontend	7
5.2	Cas 2 : Vous êtes dans l'Équipe Backend	7
5.3	Mettre à Jour Votre Branche	7
5.4	Vérifier Que Vous Êtes sur la Bonne Branche	7
6	Travailler sur Votre Branche	8
6.1	Organisation des Dossiers	8
6.2	Faire des Modifications	8
6.3	Enregistrer Vos Changements (Commit)	8
6.4	Envoyer Vos Commits sur GitHub	9
7	Créer une Pull Request (PR)	10
7.1	Quand Créer une Pull Request ?	10
7.2	Étapes pour Créer une PR sur GitHub	10
7.3	Que Se Passe-t-il Ensuite ?	10
8	Synchronisation Quotidienne	11
8.1	Mettre à Jour Votre Branche Chaque Matin	11
8.2	Gérer les Conflits de Fusion	11
8.2.1	Qu'est-ce qu'un Conflit ?	11
8.2.2	Résoudre un Conflit	11
9	Commandes Git Essentielles - Récapitulatif	13
9.1	Configuration Initiale	13
9.2	Travail Quotidien	13
9.3	Commandes de Vérification	13
9.4	Commandes Interdites	13
10	Checklist Quotidienne	14
10.1	Le Matin (Avant de Coder)	14
10.2	Pendant le Travail	14
10.3	Le Soir (Fin de Journée)	14

11 FAQ et Problèmes Courants	15
11.1 Q1 : J'ai modifié un fichier par erreur dans un autre dossier	15
11.2 Q2 : J'ai fait un commit sur la mauvaise branche	15
11.3 Q3 : Mon push est rejeté par GitHub	15
11.4 Q4 : Je ne vois pas ma branche dans VS Code	15

1 Organisation du Dépôt Git

1.1 Structure des Branches

Notre dépôt Git est organisé autour de **deux branches principales protégées** :

Branches Principales

- **main** : Version stable et finale du projet
 - Code testé et validé
 - Prêt pour démonstration/déploiement
 -  **PROTÉGÉE** - Accès restreint
- **dev** : Branche de développement principal
 - Intégration de toutes les features
 - Tests et validation avant merge vers main
 -  **PROTÉGÉE** - Accès restreint

1.2 Branches de Travail par Équipe

Chaque équipe dispose de sa propre branche pour travailler :

- **backend** : Équipe Backend (API Django, TF-IDF)
- **frontend** : Équipe Frontend (HTML/CSS/JS)
- **database** : Équipe Base de Données (Models, migrations)
- **data** : Équipe Structuration des Données (CSV, génération)

Règle Fondamentale

Vous ne travaillez JAMAIS directement sur main ou dev.

Vous créez et travaillez sur votre branche d'équipe, puis vous proposez vos changements via une Pull Request.

2 Règles du Dépôt

2.1 Règle #1 : Ne touchez pas à main et dev directement

- ✗ Ces deux branches sont **protégées**
- ✗ Vous ne pouvez pas les supprimer
- ✗ Vous ne pouvez pas y envoyer du code directement (push)

2.2 Règle #2 : Travaillez sur vos propres branches

- ✓ Exemple : **backend, frontend, database, data**
- ✓ Faites vos modifications et commits là-bas
- ✓ Testez votre code avant de proposer

2.3 Règle #3 : Créez une Pull Request (PR) quand vous êtes prêts

- 🗨 Une PR = "Voilà mon travail, est-ce qu'on peut l'ajouter dans dev ?"
- 👤 Les chefs de projet vérifient et valident
- 👍 Après validation, votre code est fusionné dans dev

3 Workflow Résumé pour l'Équipe

1. **Vous codez** sur vos branches personnelles (ex : `backend`)
2. **Vous proposez** vos changements via une Pull Request
3. **Les chefs de projet** valident et fusionnent dans `dev`
4. **Quand tout est stable**, `dev` est fusionné dans `main`

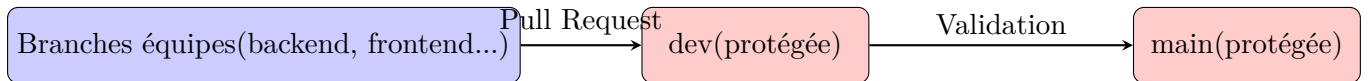


FIGURE 1 – Flux de travail Git simplifié

4 Démarrer : Cloner le Dépôt

4.1 Étape 1 : Cloner le Dépôt sur Votre Machine

Listing 1 – Dans le terminal VS Code ou PowerShell

```
# Cloner le depot
git clone https://github.com/nkoumougrinnel/ChatBot.git

# Entrer dans le dossier
cd ChatBot
```

4.2 Étape 2 : Vérifier les Branches Disponibles

```
# Lister toutes les branches (locales et distantes)
git branch -a
```

Résultat attendu :

```
* main
remotes/origin/backend
remotes/origin/frontend
remotes/origin/database
remotes/origin/data
remotes/origin/dev
remotes/origin/main
```

Comprendre les branches

- Les branches avec `remotes/origin/` sont sur GitHub (serveur distant)
- L'étoile `*` indique la branche actuelle

5 Récupérer Votre Branche de Travail

5.1 Cas 1 : Vous êtes dans l'Équipe Frontend

```
# Se positionner sur la branche frontend
git checkout frontend
```

Que se passe-t-il ?

- ✓ Git crée automatiquement la branche `frontend` en local
- ✓ Git la synchronise avec `origin/frontend` (GitHub)
- ✓ Vous êtes maintenant sur la branche `frontend`

5.2 Cas 2 : Vous êtes dans l'Équipe Backend

```
# Se positionner sur la branche backend
git checkout backend
```

5.3 Mettre à Jour Votre Branche

Important : Avant de commencer à coder, récupérez les dernières modifications :

```
# Recuperer les derniers changements
git pull origin frontend
```

(Remplacez `frontend` par le nom de votre branche)

5.4 Vérifier Que Vous Êtes sur la Bonne Branche

```
# Verifier la branche actuelle
git branch
```

Résultat attendu :

```
backend
database
* frontend    <- L'etoile indique votre branche actuelle
main
```


6 Travailler sur Votre Branche

6.1 Organisation des Dossiers

Quand vous ouvrez le projet dans VS Code, vous voyez tous les dossiers :

```
ChatBot/  
backend/      <- Backend et Database travaillent ICI  
frontend/     <- Frontend travaille ICI  
data/         <- Structuration et Database travaillent ICI  
docs/  
README.md
```

Règle de Travail

Même si vous voyez tous les dossiers, vous devez travailler **UNIQUEMENT** dans le dossier de votre équipe.

Exemples :

- Équipe Frontend Modifiez seulement les fichiers dans **frontend/**
- Équipe Backend Modifiez seulement les fichiers dans **backend/**
- Équipe Database Modifiez seulement les fichiers dans **backend/** (models, migrations)
- Équipe Data-struct Modifiez seulement les fichiers dans **data/**

6.2 Faire des Modifications

1. Ouvrez les fichiers de votre dossier
2. Codez, testez, corrigez
3. Sauvegardez vos fichiers (Ctrl+S)

6.3 Enregistrer Vos Changements (Commit)

```
# Voir les fichiers modifiés  
git status  
  
# Ajouter tous les fichiers modifiés  
git add .  
  
# Créer un commit avec un message clair  
git commit -m "Ajout interface chat + styles responsive"
```

Messages de Commit Clairs

Mauvais exemples :

- ✗ "update"
- ✗ "fix"
- ✗ "changes"

Bons exemples :

- ✓ "Ajout endpoint POST /api/chatbot/ask/"
- ✓ "Correction bug affichage feedback"
- ✓ "Import 200 Q/R catégorie Examens"

6.4 Envoyer Vos Commits sur GitHub

```
# Envoyer sur GitHub  
git push origin frontend
```

*(Remplacez **frontend** par le nom de votre branche)*

7 Créer une Pull Request (PR)

7.1 Quand Créer une Pull Request ?

- ✓ Vous avez terminé une fonctionnalité importante
- ✓ Votre code est testé et fonctionne
- ✓ Vous voulez que votre travail soit intégré dans dev

7.2 Étapes pour Créer une PR sur GitHub

1. Allez sur GitHub : <https://github.com/votre-compte/chatbot-supptic>
2. Cliquez sur l'onglet "**Pull requests**"
3. Cliquez sur "**New pull request**"
4. Sélectionnez :
 - **Base** : dev (destination)
 - **Compare** : frontend (votre branche)
5. Donnez un titre clair :

Exemple : "Frontend - Interface chat complete + feedback"

6. Décrivez vos changements :

```
## Modifications
- Ajout interface chat responsive
- Integration API /chatbot/ask/
- Boutons feedback (like/dislike)
- Page statistiques basique

## Tests effectués
- Chrome, Firefox, Safari
- Mode mobile
- Connexion API testée
```

7. Cliquez sur "**Create pull request**"

7.3 Que Se Passe-t-il Ensuite ?

Processus de Validation

1. Les **chefs de projet** reçoivent une notification
2. Ils examinent votre code
3. Ils peuvent :
 - ✓ **Approuver** et fusionner dans dev
 - 💬 **Commenter** et demander des modifications
 - ✗ **Rejeter** si le code pose problème
4. Une fois approuvé, votre code est dans **dev** !

8 Synchronisation Quotidienne

8.1 Mettre à Jour Votre Branche Chaque Matin

Avant de commencer à coder chaque jour :

```
# 1. Se positionner sur votre branche
git checkout frontend

# 2. Recuperer les derniers changements de dev
git pull origin dev

# 3. Fusionner dev dans votre branche
git merge dev

# 4. Résoudre les conflits eventuels (voir section suivante)

# 5. Envoyer la mise a jour
git push origin frontend
```

Pourquoi Mettre à Jour ?

D'autres équipes ont peut-être ajouté du code dans **dev** hier soir.
En fusionnant **dev** dans votre branche chaque matin, vous évitez les conflits majeurs et restez synchronisés avec le projet.

8.2 Gérer les Conflits de Fusion

8.2.1 Qu'est-ce qu'un Conflit ?

Un conflit arrive quand :

- Vous avez modifié un fichier
- Quelqu'un d'autre a modifié le **même fichier** au **même endroit**
- Git ne sait pas quelle version garder

8.2.2 Résoudre un Conflit

```
# Après git merge dev, si conflit :
# Git affiche : CONFLICT (content): Merge conflict in frontend/index.html
```

Étapes :

1. Ouvrez le fichier en conflit dans VS Code
2. Vous verrez des marqueurs :

```
<<<<<<< HEAD
Votre code
=====
Code de l'autre personne
>>>>>>> dev
```

3. Choisissez quelle version garder (ou combinez les deux)
4. Supprimez les marqueurs «<<<<, =====, >>>>»
5. Sauvegardez le fichier

6. Ajoutez et commitez :

```
git add .  
git commit -m "Resolution conflit merge dev"  
git push origin frontend
```

VS Code Aide Beaucoup !

VS Code détecte automatiquement les conflits et propose des boutons :

- "Accept Current Change" (garder votre version)
- "Accept Incoming Change" (garder leur version)
- "Accept Both Changes" (garder les deux)

9 Commandes Git Essentielles - Récapitulatif

9.1 Configuration Initiale

```
# Cloner le depot
git clone https://github.com/compte/chatbot-supptic.git
cd chatbot-supptic

# Voir toutes les branches
git branch -a
```

9.2 Travail Quotidien

```
# Se positionner sur votre branche
git checkout frontend

# Mettre a jour depuis dev (chaque matin)
git pull origin dev
git merge dev

# Voir les fichiers modifies
git status

# Ajouter tous les changements
git add .

# Creer un commit
git commit -m "Message clair et descriptif"

# Envoyer sur GitHub
git push origin frontend
```

9.3 Commandes de Vérification

```
# Quelle branche suis-je ?
git branch

# Voir l'historique des commits
git log --oneline

# Voir les differences avant commit
git diff
```

9.4 Commandes Interdites

À NE JAMAIS FAIRE

```
# INTERDIT sur main et dev
git push --force

# INTERDIT - Ne pas supprimer ces branches
git branch -D main
git branch -D dev
```

10 Checklist Quotidienne

10.1 Le Matin (Avant de Coder)

- ☐ Je me positionne sur ma branche : `git checkout frontend`
- ☐ Je récupère les dernières modifications : `git pull origin dev`
- ☐ Je fusionne dev dans ma branche : `git merge dev`
- ☐ Je résous les conflits éventuels
- ☐ Je vérifie que tout fonctionne (tests)

10.2 Pendant le Travail

- ☐ Je code dans MON dossier uniquement
- ☐ Je teste régulièrement
- ☐ Je fais des commits réguliers (toutes les 1-2h)
- ☐ Mes messages de commit sont clairs

10.3 Le Soir (Fin de Journée)

- ☐ Je vérifie que mon code fonctionne
- ☐ Je fais un dernier commit : `git add . && git commit -m "..."`
- ☐ J'envoie sur GitHub : `git push origin frontend`
- ☐ Si ma feature est terminée, je crée une Pull Request

11 FAQ et Problèmes Courants

11.1 Q1 : J'ai modifié un fichier par erreur dans un autre dossier

Réponse :

```
# Annuler les modifications NON commitees
git checkout -- chemin/vers/fichier

# Ou annuler TOUTES les modifications non commitees
git reset --hard
```

Attention

git reset -hard supprime TOUTES vos modifications non sauvegardées !

11.2 Q2 : J'ai fait un commit sur la mauvaise branche

Réponse :

```
# Annuler le dernier commit (mais garder les modifications)
git reset --soft HEAD~1

# Se positionner sur la bonne branche
git checkout ma-vraie-branche

# Refaire le commit
git add .
git commit -m "Mon message"
```

11.3 Q3 : Mon push est rejeté par GitHub

Message d'erreur :

```
! [rejected] frontend -> frontend (non-fast-forward)
```

Cause : Quelqu'un a push sur la même branche pendant que vous travailliez.

Solution :

```
# Recuperer les changements
git pull origin frontend

# Resoudre les conflits eventuels

# Re-pousser
git push origin frontend
```

11.4 Q4 : Je ne vois pas ma branche dans VS Code

Solution :

```
# Mettre a jour la liste des branches
git fetch origin

# Lister toutes les branches
git branch -a

# Se positionner sur la branche
```



```
git checkout ma-branche
```

Conclusion

Résumé en 3 Points

1. **Travaillez sur VOTRE branche** (backend, frontend, database, data-struct)
2. **Synchronisez-vous CHAQUE MATIN** avec dev
3. **Proposez vos changements** via Pull Request quand c'est prêt

Document préparé par :
NKOUMOU TJADE
Chef de la Division de Programmation
Club Informatique SUP'PTIC

Bonne collaboration à tous !